2024 20th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT) | 979-8-3503-6944-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/DCOSS-IoT61029.2024.00012

Local Differential Privacy for Smart Meter Data Sharing with Energy Disaggregation

Yashothara Shanmugarasa*†, M.A.P. Chamikara†, Hye-young Paik*, Salil S. Kanhere*, Liming Zhu†
*University of New South Wales, Australia
†CSIRO's Data61, Australia
E-mail: {y.shanmugarasa, h.paik, salil.kanhere}@unsw.edu.au,
{yashothara.shanmugarasa, chamikara.arachchige, liming.zhu}@data61.csiro.au

***Abstract*—Smart meter data with energy disaggregation can be used to identify energy usage of appliances, offering valuable insights for consumers and energy companies to better manage energy. However, these techniques also present privacy risks, such as the potential for behavioral profiling. Local differential privacy (LDP) methods provide strong privacy guarantees with high efficiency in addressing privacy concerns. Exiting LDP methods primarily focus on securing aggregated energy data, ignoring the streaming nature of smart meter data and individual appliance data. In this paper, we propose a novel LDP approach (named LDP–$Energy$) to facilitate the sharing of appliance-level energy data *over time* while not revealing *individual users' appliances and their usage* patterns. By incorporating a sliding window concept, LDP–$Energy$ is able to efficiently handle the streaming smart meter data. Our evaluations indicate that LDP–$Energy$ performs efficiently compared to other methods, striking a proper balance between privacy and data utility for effective analysis.**

I. Introduction

Smart meters digitally track real-time energy consumption at a particular location (e.g., a house, a building) and transmit the corresponding data to energy companies for analysis [1]. Smart meters offer various benefits, including accurate energy usage billing and improved grid operation stability [1].

Energy disaggregation (ED) refers to the process of determining the energy usage of individual appliances (e.g., within a building). Users could share ED data with energy companies or third parties to gain insights into individualized appliance usage data. For instance, users could compare their energy consumption of particular appliances with households in the area. From the viewpoint of energy companies, collecting ED data from users can assist in identifying energy usage trends, segmenting consumers based on energy usage patterns, and troubleshooting issues in real-time [2].

However, sharing ED data raises privacy concerns. For instance, they could enable real-time monitoring of household activities, appliance usage, behavioral profiling , and identity theft [3]. Also, ED data reveals the list of appliances the user owns, which may not be desirable for users if they do not want to reveal certain types of appliances, such as medical aids.

The literature shows various privacy-preserving techniques to reduce the privacy risks in smart meter data sharing [1]. Approaches using differential privacy (DP) [4] are favored over other privacy preservation techniques, such as cryptographic methods. Cryptographic approaches can be computationally complex and suffer from performance degradation [5].

DP schemes achieve data protection by introducing perturbation techniques (e.g., applying calibrated noise or randomizing data values) to the data. In global differential privacy (GDP), a trusted curator would perform perturbation. In local differential privacy (LDP), the data owners are responsible for applying the techniques before sharing. It is shown that LDP offers stronger privacy guarantees than GDP at the expense of slightly lower utility (accuracy and usefulness of data analysis results) [6].

To the best of our knowledge, the existing DP work has been applied to aggregated energy usage data (e.g., calculating total or average energy consumption) and is not directly applicable to disaggregated data obtained through ED [1]. ED data sharing involves revealing a user's appliance list along with individualized appliance data. Also, smart meter data are streaming (i.e., continuous); as such, we consider ED data to be a type of streaming data that is shared periodically in an ongoing manner. With these characteristics in mind, in utilizing LDP schemes for ED, we identify the following technical gaps in the existing literature.

- Existing LDP techniques (e.g., Laplace [7], Geometric [8], Gamma [9]) can protect individual appliance data (i.e., calibrated noise can be added to each appliance data separately). However, none of the methods we investigated can obscure the list of appliances a user owns. We need a solution that hides the appliance list while still protecting individual appliance data in the list.
- The management of "privacy budget" (a threshold for allowable privacy loss) in existing LDP methods assumes that the data are shared once. As in ED, the data are shared in an ongoing manner; we need a solution that considers privacy loss over a time period (i.e., multiple time windows).

In this paper, we introduce a novel LDP protocol, LDP–$Energy$, for sharing of ED data. The novelty of our work is summarised as follows:

- We propose a new LDP method that uniquely employs Randomized Response (RR) [10] and Global Appliance List (GAL) (further discussed in IV-A). The method achieves privacy-preserving sharing of ED data that protects the appliance list and individual appliance data while maintaining the utility of global insights.
- We adopt a sliding window approach for managing the

2325-2944/24/$31.00 ©2024 IEEE
DOI 10.1109/DCOSS-IoT61029.2024.00012

Authorized licensed use limited to: University of Western Australia. Downloaded on July 11,2026 at 19:08:58 UTC from IEEE Xplore. Restrictions apply.

overall privacy budget over time. Our approach builds upon "w-event privacy" [11] for handling ED stream data, as opposed to [11] method, which used the corresponding technique primarily for frequency estimation of categorical data, our method works on numerical energy data.

We present formal proofs of our methods to show that they can meet the privacy budget constraints defined within LDP. To show utility, we implement our solution through a data analysis scenario named "Top-k appliances and their usage patterns," which looks into k number of appliances that consume the most energy. Using a real-world dataset, we could show that $LDP\text{–}Energy$ identifies the Top 8 appliances out of 10, demonstrating high accuracy.

II. Related Work

Various privacy-preserving approaches for smart meter data exist in the literature, encompassing cryptographic methods, privacy with demand shaping and load scheduling, and statistical approaches [1]. Cryptographic techniques, like homomorphic encryption (HE), secure data by encrypting it, preventing unauthorized access. Secure multiparty computation (MPC), such as secret sharing, enables secure function evaluation on distributed private data among untrusted parties. Cryptographic methods face challenges like being computationally complex, vulnerable to Sybil attacks, and involving higher communication costs in resource-constrained environments [5].

Another privacy-preserving approach specific to smart meters is battery load balancing by charging/discharging energy storage (external batteries), which can hide energy consumption time and appliance load signatures [12]. However, it also has issues such as short battery lifespan, cost of installing large batteries, and environmental impact [7].

Common statistical methods to secure smart meter privacy include k-anonymity, aggregation, and DP [1]. DP, known for its reduced computational overhead and robust privacy protection, has been implemented using noise types like Gamma [9], Laplace [7], [13], and geometric [8], with Laplace being prevalent. Point-wise DP [9] and the Multi-Armed Bandit DP algorithm [13] are proposed as real-time and cost-effective DP strategies. Gai et al. [5] presented a scheme using randomized responses to estimate total or average power consumption while preserving LDP.

Existing privacy-preserving approaches for smart meter data mainly concentrate on aggregated statistics for billing purposes rather than leveraging disaggregated data for value-added services. This paper addresses the gap by focusing on ED data of individual appliances, essential for services like identifying the impact of appliances on energy consumption, consumer segmentation, and anomaly detection [14]. Existing methods are not directly applicable to ED data, as they may reveal individual appliance details. Current LDP methods don't address the streaming nature of smart meter data, lacking time-window processing to maintain privacy over time.

III. Preliminaries

A. Energy Disaggregation over Appliances

A smart meter measurement, V_t represents the total energy consumption at a particular location (e.g., a house or a building) measured at a time t. Energy disaggregation (ED) refers to identifying the energy consumption of individual appliances contributes to V_t. We model this as follows. Suppose a location (e.g., a single household) has n appliances. Then $A = \{a_1, a_2, \ldots, a_n\}$, the energy consumption of the appliances $\{[a_{1t}, v_{1t}], \ldots, [a_{nt}, v_{nt}]\}$ at time t can be represented as denoted in Eq. (1). The variable v_{it} represents the energy consumption of a_i, while E_t represents the error during ED.

$$V_t = \sum_{i=1}^{n}((a_{it} \times v_{it}) + E_t) \quad \begin{matrix} a_{it}=1 \text{ if appliance } a_i \text{ is on at } t; \\ 0 \text{ if appliance } a_i \text{ is off at time } t. \end{matrix} \quad (1)$$

B. Local Differential Privacy (LDP)

LDP is a privacy preservation technique where data owners can apply randomization or noise to their data before sharing them with a third party. If a randomized algorithm $\mathcal{A}$ satisfies Eq. (2), then it provides ε-LDP.

Definition III.1. An algorithm $\mathcal{A}$ satisfies ε-LDP for any user if for any two data values v_1 and v_2, and for any output Q within the range of outputs of algorithm $\mathcal{A}$ ($Q \subseteq Range(\mathcal{A})$), the following Eq. (2) holds (ε is the privacy budget and it represents the degree of privacy) [10]:

$$Pr\left[\mathcal{A}(v_1) \in Q\right] \leq e^{\varepsilon} \times \Pr[\mathcal{A}(v_2) \in Q] \quad (2)$$

where the probability is taken over the random measure implicit in $\mathcal{A}$.

C. Randomized Aggregatable Privacy-Preserving Ordinal Response (RAPPOR)

RAPPOR is a technique proposed in [10] to provide strong privacy guarantees for users participating in crowdsourcing data collection. RAPPOR encodes an input data instance (i.e., user responses) into a binary string, B, which is randomly perturbed before being sent to a data processing server. A binary string, B, is composed of $d-1$ zeros and one 1 at a specific position, v. Randomization preserves privacy by randomizing the 0s and 1s in the binary string B, creating a new string, B'. During the randomization process, the true value of an input bit-string B is preserved with a probability of p, as in Eq. (3) where Δf is the sensitivity of a function f, which is the maximum impact that a single individual can exert on a function f In RAPPOR's encoding process, sensitivity (Δf) can be defined as the maximum difference between two adjacent encoded bit strings, $B(x_i)$ and $B(x_{i+1})$. This difference is limited to two bits.

$$p = \frac{e^{\frac{\varepsilon}{\Delta f}}}{1 + e^{\frac{\varepsilon}{\Delta f}}} = \frac{e^{\frac{\varepsilon}{2}}}{1 + e^{\frac{\varepsilon}{2}}} \quad (3)$$

Authorized licensed use limited to: University of Western Australia. Downloaded on July 11,2026 at 19:08:58 UTC from IEEE Xplore. Restrictions apply.

D. Unary Encoding and Optimized Unary Encoding

RAPPOR employs unary encoding (UE) [15] to transform the input instance y_i into a binary vector B consisting of d number of bits. UE sets a specific bit at position v in B to 1, and all others to 0, then perturb a bit, $B[i]$, into a perturbed bit B' [I].

UE provides ε-LDP [10]. Specifically, for any bit positions v_1 and v_2 (of the encoded inputs x_1 and x_2, respectively) in a d-bit vector, and an output B with sensitivity = 2, UE ensures ε-LDP when the values of p and q are given by $p = \frac{e^{\varepsilon/2}}{1+e^{\varepsilon/2}}$ and $q = \frac{1}{1+e^{\varepsilon/2}}$. The highest value of this ratio occurs when v_1 is 1 and v_2 is 0.

The Optimized Unary Encoding (OUE) method improves upon UE by perturbing 0s and 1s differently. There are more 0s than 1s (usually only one 1) in a long binary vector, B. OUE reduces the chance of perturbing 0 to 1 ($p_{0\to1}$) by allocating more budget to transmit 0 bits in their original state. By setting $p = 1/2$ and $q = \frac{1}{1+e^{\epsilon}}$ [15], OUE guarantees ϵ-LDP for values with sensitivity equal to 2 [16].

E. Adaptive Privacy Budget over Time

The authors in [11] proposed adaptive privacy budget division methods within the LDP-IDS framework for streaming data, addressing the limitations of concealing single events in data streams. Introducing "w-event privacy," the concept ensures ϵ-DP for time windows comprising w consecutive instances, offering a more practical approach for streaming data for user privacy. Further details are in Appendix B.

IV. Methodology

An energy consumption data-sharing framework that utilizes $LDP\text{–}Energy$ is shown in Figure 1. Figure 1a depicts the processes within $LDP\text{–}Energy$, which is implemented and run on the user side. First, $LDP\text{–}Energy$ takes ED data from appliances and applies the following three steps: (1) Mapping the individual appliances' energy consumption to a given range of energy levels (step 1 in Figure 1a), (2) Encoding energy consumption levels, generating 1-D vector with all appliance details, and adding randomization to enforce DP (step 2 in Figure 1a), and (3) Maintaining the budget composition of a data stream using an adaptive budget division methods (step 3 in Figure 1a). Step 3 determines the privacy budget for randomization in each timestamp of step 2.

A. The Rationale

The appliance list shared through ED could identify names or types of appliances a user owns. As mentioned earlier, none of the LDP techniques consider obscuring the list. We introduce GAL, a pre-arranged list of appliances in the system. Users will report on all appliances in the list irrespective of their actual presence in their home. This allows obscuring the actual appliances present. However, though we refine existing noise addition techniques by incorporating GAL and share the data after adding noise to available and non-available appliances in GAL, there is still a risk of revealing appliances at home. This vulnerability emerges because non-available appliances from GAL often have zero or lower values even after adding noise, making them identifiable as appliances not present at the user's home. The potential exposure of the appliance list may happen even when employing randomization techniques to individual appliances separately due to insufficient randomization. Hence, prior to randomization, our approach concatenates all individual appliance data and employs bit-wise randomization to effectively conceal both the list of appliances and the energy consumption. However, Bit-wise randomization can apply to only categorical data; we introduce a preprocessing step that transforms them into discrete values via mapping and quantization before randomization.

Secondly, existing LDP methods assume that the data are shared once. As in ED, the data are shared in an ongoing manner; we adopt a sliding window approach, which builds upon "w-event privacy" [11] to guarantee privacy over time.

B. $LDP\text{–}Energy$ Overview

In $LDP\text{–}Energy$, a service provider, as shown in Figure 1b, initiates the process by determining parameters such as the appliance list, privacy budget (ε), data collection time interval, energy level ranges, and window size. The hyperparameters are data-independent and predetermined by service providers based on prior knowledge.

The process on the user side starts with mapping to limit the disclosure of individual appliances' energy values while also enabling high efficiency with concurrency. Next, the energy levels are converted into binary vectors for randomization with high privacy and utility.

Finally, the service provider completes a data collection round by aggregating the randomized data from all users. To showcase the utility of randomized data with $LDP\text{–}Energy$, we examine a practical use case: a household sharing smart meter data with a service provider, and the service provider identifies the top-k most frequently used appliances from the given user data.

C. Mapping the Individual Appliances' Energy Consumption into Range Levels

Smart meters generally measure energy consumption within a certain range [0, max] at a specific time t. The 'max' value is selected based on prior knowledge of energy consumption patterns, independent of the data. It is reasonable to assume that energy data submitted by honest users will fall within the range. In this step, we map energy values (numeric) into distinct energy levels L by dividing the maximum range of energy consumption [0, max] into smaller, distinct ranges. As appliances have varying energy consumption levels, different ranges were used to map the data. Specifically, we create a set of distinct ranges for energy levels L, r = {(0,r_1):l_1, (r_1, r_2):l_2,..., (r_d, max):l_d} where the number of energy levels L is equal to d, we label each range [r_i, r_j] as a distinct energy level L = {l_1, l_2, ..., l_d}.

For a user u_i, the energy consumption (v_i) of an appliance a_i is mapped to the corresponding range (r_i, r_j), generating the energy level l_i. This process is iteratively applied to all

Authorized licensed use limited to: University of Western Australia. Downloaded on July 11,2026 at 19:08:58 UTC from IEEE Xplore. Restrictions apply.

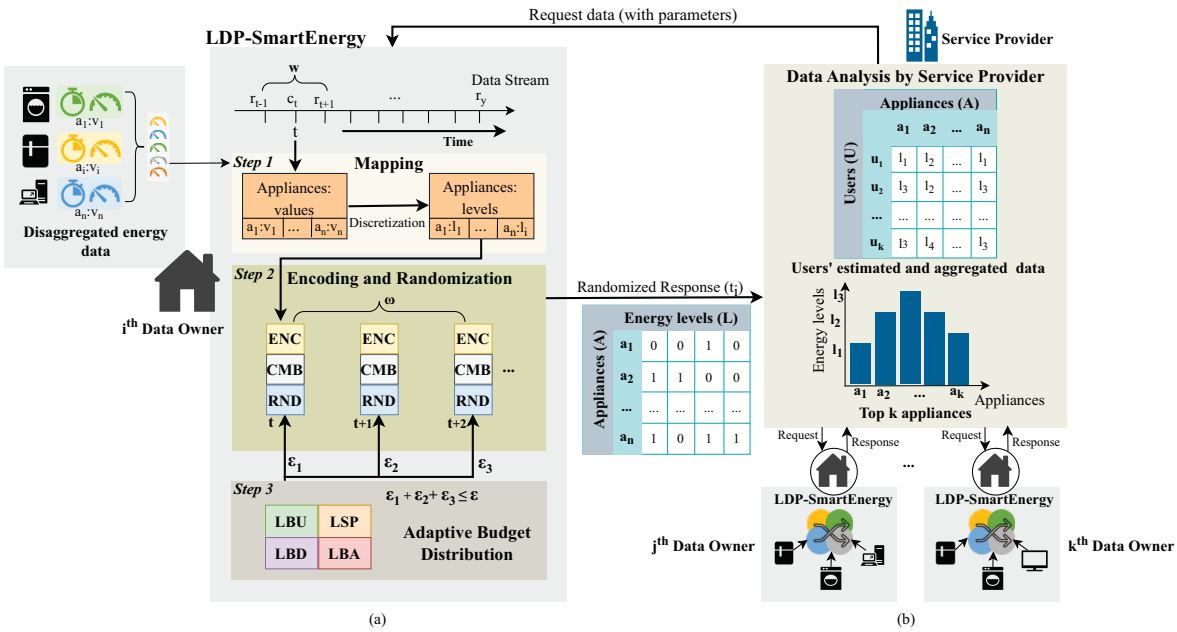


Figure 1: The architecture of *LDP–Energy* (a): On the user (data owner) side, (b): On the service provider side. $\{r_{t-1}, c_t, r_{t+1}, \ldots, r_y\}$: Energy consumption data stream, c_t: Data of the current timestamp, $\{a_1, v_1, \ldots, a_n, v_n\}$: appliances with their corresponding energy values, $\{a_1, l_1, \ldots, a_n, l_i\}$: appliances with their corresponding mapped energy levels, **ENC**: Encoding, **CMB**: Combining all appliances encoded data, **RND**: Randomization, $\varepsilon_1, \varepsilon_2, \varepsilon_3, \varepsilon$: Privacy budgets

Algorithm 1: *LDP–Energy* user-side algorithm

Input : {<a_1,v_1>, …, <a_n, v_n>} ← n no. of appliances and corresponding energy values
{(0,r_1):l_1, …, (r_d, max):l_d} ← energy ranges and corresponding levels L
ε ← total privacy budget
w ← window size

Output: {B_1, …, B_i} ← 1-D vector of randomized energy consumption over i time periods for all appliances

Step I: Mapping the Individual Appliances' Energy Consumption into Range Levels (refer to Section IV-C)
for *each timestamp* t **do**
  for *each* <a_i,v_i> **do**
    for *each* (r_d, r_i):l_i **do**
      if $v_i > r_d$ *and* $v_i \leq r_i$ **then**
        return <a_i,(l_i)>
    end for
  end for
  return {<a_1,l_1>, …, <a_n, l_i>}
end for
Step II: Encoding, Vector Generation and Randomization
Declare, $m = (d \times n)$, 1-D binary vector B length of m
for *each timestamp* t **do**
  Encode {<a_1,l_1>, …, <a_n, l_i>} to d bits of binary vector v (refer to Section IV-D1)
  for *each appliance vector* v **do**
    Append v to B
  end for
  return B
  Calculate ε and randomize B into RND from part III
end for
Step III: Adaptive Budget Division
for *each timestamp* t **do**
  Approach 1: LDP budget uniform (LBU)
  $\varepsilon_1 = \varepsilon / w$
  return RND = randomize B with ε_1
  Approach 2: LDP sampling method(LSP)
  if t == $samplingtimestamp$ **then**
    $\varepsilon_2 = \varepsilon$
  else
    $\varepsilon_2 = 0$
  end if
  return RND = randomize B with ε_2
  Approach 3 & 4: Local budget distribution (LBD) or Local budget absorption (LBA)
  // Sub Mechanism $M_{t,1}$, $M_{t,2}$, and $M_{t,3}$
  refer to Algorithm 3
end for

appliances listed by the service provider in GAL even if they don't have the appliance), producing a set of energy levels for the user's energy consumption: a_1:l_1, a_2:l_2,…, a_n:l_i, where n is the total number of appliances.

D. Encoding, Vector Generation and Randomization

After mapping the appliances' energy values to energy levels, the discretized energy levels are encoded into binary vectors using binary encoding. This step can also reduce com-

Authorized licensed use limited to: University of Western Australia. Downloaded on July 11,2026 at 19:08:58 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: LDP–$Energy$ server-side algorithm

Input : {<u_1,B_1>, ..., <u_k,B_k>} ← k number of users & corresponding concatenated 1-D flattened vector with energy levels of appliances list
ε ← maximum total privacy budget
n ← number of appliances
d ← number of energy levels
Δf = 2 ← each appliance sensitivity

Output: {<a_1,l_i,c_i>, ..., <a_1, l_i, c_j>, ..., <a_n,l_i,c_i>, ..., <a_n, l_i, c_j>} ← Aggregated and estimated energy levels of the appliance list

{ <a_1,p_1>, ..., <a_n,p_n>} = Decode the B into appliance-wise energy levels by dividing B by d
for *each* <a_i,p_i> **do**
  for *each user* u **do**
    Define p = 0.5, $q = \frac{1}{(1+e^{\frac{\varepsilon}{\Delta f/2}})}$
    // Aggregate and estimate energy levels
    $sums$ = sum p_i of each u
    for *each* $sums$ y **do**
      return $aggregatedPerturbedValues$ = $(y - d * q)/(p - q)$
    end for
  end for
  return {<a_i,l_i,c_i>, ..., <a_i, l_i, c_j>}
end for
return {{<a_1,l_i,c_i>, ..., <a_1, l_i, c_j>}, ..., {<a_n,l_i,c_i>, ..., <a_n, l_i, c_j>}}

putation and communication costs in data transactions. The encoded energy levels for all appliances are then concatenated and flattened into a single 1-D vector, which is randomized to ensure privacy in the resulting binary vector.

1) Binary Encoding: Each appliance's energy value is converted into a binary value with d number of bits, where d is the number of energy levels. Each bit of the binary vector represents a specific energy level l_i. If an appliance's energy value falls within the range of a certain energy level l_i (determined in step 1), a binary value will have a '1' in the corresponding bit position of l_i, and all other positions will have '0'. For example, suppose an appliance's energy consumption belongs to l_1. In that case, the corresponding encoded value will be '$[0, 0, 0, 0, 0, 0, 0, 0, 0, 1]$' where the first-bit position is '1' (if the energy value falls in l_2, the second-bit position will be '1') while the rest are '0's, given that d is 10.

2) 1-D Vector Generation: To prepare for the randomization process, all the binary values of the encoded appliance's energy consumption are concatenated into a single, flattened, long binary 1-D vector (B). B has a length of $m = d \times n$, where d is the number of energy levels and n is the number of appliances.

This step is vital to ensure that distributing the privacy budget among all appliances does not negatively impact utility. Following the composition property of DP, when addressing each appliance individually, a small fraction of the privacy budget is allocated to randomizing the binary value of each appliance, potentially reducing utility. However, by consolidating all encoded values of appliances into a single extended binary 1-D vector, the total privacy budget is fully utilized for all appliances. This approach not only enhances utility but also improves communication efficiency.

3) Randomization: In the randomization step, we select OUE over Unary Encoding (UE) (see Section III-D) to randomize B to maintain a high utility of the randomized data, as UE can introduce undesirable levels of randomization due to the high sensitivity of the inputs when there are many appliances. The OUE approach randomizes the encoded bit values by flipping them from 0's to 1's or vice versa with a certain probability. This step results in a binary vector that may contain multiple 1s and 0s due to the randomization process, adding a layer of privacy protection to the encoded values. The probability of randomization, denoted as p, can be determined as $p = \frac{1}{2}$ and $q = \frac{1}{(1+e^{\frac{\varepsilon}{2n/2}})}$, where the probability of perturbing 0 to 1 ($p_{0\to1}$) is reduced by utilizing OUE. The sensitivity of the encoded value for an individual appliance is 2, as the maximum possible difference of two binary inputs (B_1 and B_2, please refer to Section III-C) is 2 bits. Hence, when all appliance binary strings are concatenated into B, the sensitivity becomes $2\times n$. Theorem 1 (please refer to Appendix A) shows that LDP–$Energy$ satisfies DP.

E. Adaptive Budget Distribution

Since energy data are collected over time, there is a potential risk of gradual privacy leakage over time. So, as shown in Figure 1a, in step 3 of the workflow, the adaptive budget division methods LBU, LSP, LBD, and LBA are used to calculate the privacy budget for each timestamp within a window to enforce DP on smart meter stream data over time.

LBD/LBA dynamically selects between publication and approximation based on the dissimilarity dis and potential publication error err at each timestamp. If $dis < err$, the approximation is chosen, the current budget is set to 0, and the previous release is published directly. Otherwise, the perturbation strategy is used with the allocated budget ($\varepsilon_{t,2}$) for the randomization step (step 2 in Figure 1).

F. Data Analysis Scenario by Service Provider

We use "energy consumption aggregation and estimation of the Top-k commonly used appliances" as a data analysis scenario performed by the service provider. As per the agreed-upon data collection interval, users send randomized binary vectors to the service provider. After receiving perturbed data from all users, the service provider aggregates and estimates energy values for the top-k appliances. LDP–$Energy$ employs LDP for each user, assuming independence among data instances (different users). The final privacy budget consumption is the maximum of all individual privacy budgets. The perturbed B string is then processed to determine energy consumption levels (l_i) for the top-k appliances by counting the frequency of each level across all users for a given appliance.

G. The Algorithm

The exact steps for conducting LDP on ED data using LDP–$Energy$ are outlined in Algorithm 1. Algorithm 1 follows the detailed methodology discussed in Section IV. Algorithm 2 outlines the process of server-side aggregation and estimation of energy levels for the top-k appliances from k users.

Authorized licensed use limited to: University of Western Australia. Downloaded on July 11,2026 at 19:08:58 UTC from IEEE Xplore. Restrictions apply.

V. Results and Discussion

In this section, we comprehensively evaluate the performance of LDP–$Energy$using the IDEAL Household Energy Dataset [17] and synthetic datasets to test LDP–$Energy$ against various data distributions. We employ multiple measurements (e.g., different window sizes, epsilon values, and levels) to support this evaluation.

A. The datasets

The IDEAL dataset consists of detailed energy data from 39 households in the UK, covering usage data for 15 individual electrical appliances. This dataset was chosen for its extensive coverage of appliances and a reasonable number of users compared to others. To overcome its size limitation for the LDP approach, we augmented the dataset by synthesizing additional data points, emulating a sample size of 1000 users. The augmentation maintained a consistent pattern with the original IDEAL dataset, considering appliance-wise mean, standard deviation, and lower-upper bounds of energy consumption for each user [18]. Focusing on appliance-wise patterns allowed us to capture individual usage patterns more accurately, preserving users' unique energy consumption characteristics. This approach, validated in studies [19], ensures authenticity and the capability of preserving the quality of real data.

Next, the synthetic dataset was generated with varying distributions within the energy value range of [0, 3000] Watts. We chose this range based on our observations of the IDEAL data distribution. The generated data included normal, uniform, and skewed (left and right) distributions to simulate varying energy consumption patterns with 10,000 simulated users.

B. Experiments for Utility Analysis

We quantitatively assessed the effectiveness of LDP–$Energy$ by comparing the aggregated perturbed data from all users to their corresponding true appliance energy levels using the Kruskal-Wallis test [20]. This non-parametric test is suitable for non-normally distributed datasets, such as energy data, as it makes no assumptions about specific data distributions, and it is used when the data violates the assumptions of normality and equal variance. It assesses the similarity of underlying distributions between true and perturbed histograms.

1) Experiments with IDEAL dataset: In the experiments, we use a default setup with 1000 simulated users, 15 appliances, and 10 quantization levels. Hyperparameters, chosen independently from data, are typically defined by service providers based on prior knowledge. In real scenarios, service providers select energy level ranges based on experience and prior data analysis. Our experiments cover a wide range of levels, regardless of the data dynamics.

Data are distributed among simulated users and shared daily within a 3-day sliding window. We focus on 15 appliances, reflecting the common household possession range of 10 to 17 appliances [21]. Studies show that most energy usage comes from around 10-12 appliance categories, accounting for nearly 99% of total consumption.

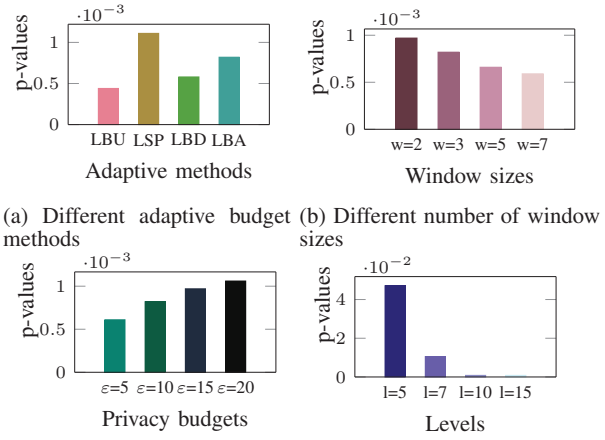
First, we assessed LDP–$Energy$ using adaptive budget division methods (LBU, LSP, LBD, LBA) with a privacy budget of 10 (a higher budget was chosen as we needed to maintain the utility of 15 appliances in a window and our dataset is relatively small in size). In our scenario, allocating a maximum of $\varepsilon = 10/15 = 0.66$ for each appliance, on average, is reasonable. This allocation aligns with the approach used by Apple as outlined in their DP implementation [22], where they utilize a ε ranging from 4 to 8 for their products with millions of users. Given our dataset of 1000 users, 15 appliances, and a window size of 3, $\varepsilon = 10$ is reasonable.

In Figure 2a, averaged p-values from the Kruskal–Wallis test for LBU, LSP, LBD, and LBA are presented. The p-values were calculated for 15 appliances over 10 iterations. We computed p-values for each appliance by comparing the true and perturbed data and then calculated the average p-value for all appliances. The results were visualized in a graph (refer to Figure 2a). In most cases, the p-values are less than 0.05, indicating less similarity. Given our small, randomized dataset, this is expected. DP fundamentally requires a large amount of data from many users for a better approximation. However, when the number of levels is 5 (Figure 2d), most appliances exhibit p-values greater than 0.05, suggesting a similar pattern between perturbed and true data. Moreover, top-k retrieval on perturbed data yields a promising average hit rate of 5.7, affirming our method's performance (Section V-B1a).

The LSP method outperforms other methods in the given dataset. This can be attributed to the characteristics of the dataset, which contains numerous instances of zero values due to the absence of energy consumption for many devices over time. LSP maximizes budget usage by sampling other releases in the dataset, capitalizing on minimal variation across neighboring timestamps in the IDEAL dataset. However, LBD and LBA outperform LSP in our synthetic dataset (Figure 4) due to their data-dependent sampling, dynamic budget allocation, and approximation strategy. The LBA also performs well in the IDEAL dataset, closely matching LSP's performance. Its effective budget allocation across multiple publications, starting with a uniform distribution and followed by dynamic absorption, maintain performance, especially with larger window sizes (Figure 2b). Moreover, the LSP displays minimal variation across configurations, indicating a tendency to sample timestamps without considering underlying data distribution, potentially missing diverse energy consumption patterns. Consequently, LBA was chosen as our default method for subsequent experiments to assess the influence of other parameters on utility.

We conducted the utility analysis using the same configuration, except for the window size w, which we incremented from 2 to 7. Figure 2b shows that a larger window size gives lower p-values, indicating more dissimilarity. The results are consistent with our theoretical utility analysis, as the privacy budget (ε) is divided by w in LBD and LBA approaches. Besides, Figure 2c is consistent with the DP process, where high ε values tend to decrease dissimilarity. Figure 2d shows that when the number of levels is 5, for most of the appliances, we

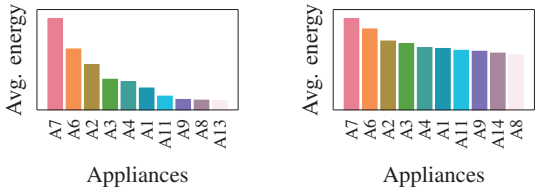
Authorized licensed use limited to: University of Western Australia. Downloaded on July 11,2026 at 19:08:58 UTC from IEEE Xplore. Restrictions apply.



(a) Different adaptive budget methods (b) Different number of window sizes

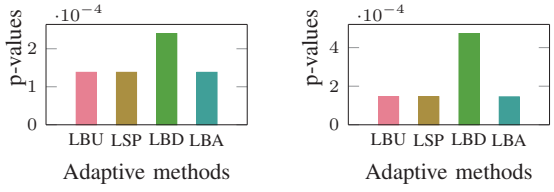
(c) Different privacy budgets (d) Different number of levels

Figure 2: The utility analysis: the average p-values for all appliances of Kruskal–Wallis test in IDEAL dataset. Except for the variations in specific scenarios, the default configuration: w = 3, adaptive method = LBA, ε = 10, l = 10



(a) True data (b) Perturbed data

Figure 3: Top-k appliance distribution



(a) Skewed (left) distribution (b) Skewed (right) distribution

Figure 4: The Kruskal–Wallis similarity test in positive and negative skewed distributions (in synthetic data)

achieve a p-value greater than 0.05, indicating a similar pattern between the perturbed and true data. Increasing the number of levels significantly reduces the utility of our approach, as the probability of falling into different levels increases, but the privacy budget (ε = 10) remains unchanged. This means there is a certain probability of flipping the energy levels from 0 to 1 or vice versa. Further, as the number of levels increases, there is more randomization over many levels, leading to a decrease in utility.

*a) **Service Provider-Side Application of Perturbed Data**:* In addition to the Kruskal-Wallis tests, we evaluated our approach by applying service-provider-side post-processing applications, such as top-k appliance inference, and analyzing appliance usage patterns (discussed in Appendix C1).

Our application involves calculating the top-k appliances within an aggregated dataset comprising data from all users. We computed the total energy consumption for each appliance over the experimented days (30 days) and sorted the list in descending order to retrieve top-k appliances. Figures 3a and 3b showcase the true and perturbed data results. Despite having limited data and operating in a constrained environment, our approach showed promising results in predicting the top 10 appliances within 15 appliances. In our evaluation (please refer to Figures 3a and 3b), our approach successfully identified 8 of the top 10 appliances, with minimal rank changes for the rest. Using the hit rate definition (*the number of times the top-k appliances are accurately identified with the correct rank / total count*), our approach achieved a hit rate of 8/10. To obtain a more consistent average hit rate, we reran our algorithm 100 times and observed an average hit rate of 5.7 (median: 6), ranging from 4 to 8.

2) Experiments with Synthetic Data: In the experimental evaluation utilizing synthetic data of varying distributions, the default configuration was utilized, which consists of a total of 10,000 simulated users, 15 distinct appliances, 10 quantization levels, a privacy budget (ε) of 2 , and a window size of 2.

We analyze the utility of Skewed positive and negative distributions in Figure 4. Due to space constraints, we relocated the utility analysis of normal and uniform distributions to Appendix C2, Figure 6. In evaluating LDP–$Energy$'s utility for various distributions (Figure 4), LBU, LSP, LBD, and LBA methods demonstrated similar average performance, with notable differences in skewed distributions as utilizing LBD could potentially save the privacy budget when releases similar to past releases occur. The LBD method outperforms LBA due to the smaller window size in our approach, which was chosen because of limited data. This enables LBD to maintain a reasonable privacy budget ε with a slower exponential decrease over the window, given the smaller size.

The experiments reveal that LBD and LBA methods perform optimally across various distributions, while LBU and LSP excel for uniform and normal distributions with minimal data fluctuations. However, since energy data may exhibit fluctuations and not strictly adhere to specific distributions, employing LBD (for small windows) or LBA (for large windows) is preferable to balance utility and privacy preservation.

C. Benchmarking

Existing methods often concentrate on applying DP to aggregate total energy usage data; but face limitations when extended to individual appliance-level data. Our approach uniquely integrates appliance-wise energy data, streaming data, and top-k appliance inference, making direct comparisons with existing approaches challenging. To ensure a fair assessment, we benchmarked against established LDP algorithms, such as Laplace, Gamma, Gaussian, and Exponential mechanisms [1], [7], [9]. For a fair comparison, we implemented these approaches for disaggregated data, as previous research has predominantly emphasized total aggregation.

In all mentioned benchmarking approaches, a fixed privacy budget ($\varepsilon = 1$) is applied at each time, unlike the adaptive budget allocation in LDP–$Energy$. This differentiation arises

Authorized licensed use limited to: University of Western Australia. Downloaded on July 11,2026 at 19:08:58 UTC from IEEE Xplore. Restrictions apply.

because existing LDP methods tailored for protecting smart meter privacy don't account for the streaming characteristics of energy data. The benchmarking methods allocate a higher privacy budget for each appliance than our approach, which allocates a privacy budget of 10 for a 3-day window, considering 15 appliances. Table I quantitatively compares the benchmarking and LDP–$Energy$ approaches using the IDEAL dataset regarding time complexity, computational complexity, and hit rate, averaged over 100 iterations.

Table I: Quantitative and qualitative comparison of LDP–$Energy$ against the existing benchmarking approaches

Approach	Time complexity	Computational complexity	Average hit rate	Stream Process
Laplace	0.06ms	Client: $O(n)$, service provider: $O(n^2)$	1.98	No
Gamma	0.11ms	Client: $O(n)$, service provider: $O(n^2)$	2.09	No
Gaussian	0.08ms	Client: $O(n)$, service provider: $O(n^2)$	2.075	No
Exponential	0.09ms	Client: $O(n)$, service provider: $O(n^2)$	2.1	No
LDP-Energy	0.009ms	Client: $O(n^2)$, service provider: $O(k)$	5.7	Yes

In terms of time complexity, initial experiments involved sequential noise addition to individual appliances. To ensure fairness, concurrent implementations were employed, matching the number of appliances (n = 15). Table I shows the time taken for concurrent noise addition by benchmarking approaches. Even in concurrent scenarios, LDP–$Energy$ exhibits lower time complexity than other methods. Benchmarking mechanisms handle data streams from appliances independently, potentially leading to inefficiencies due to a sophisticated structure, as per the composition theory.

In terms of computational complexity, our algorithm exhibits a user-side complexity of $O(n^2)$, while the service provider-side complexity is $O(k)$. The user-side complexity stems from mapping real number energy consumption to predefined range levels and performing budget calculations involving addition, subtraction, and multiplication in loops for all appliances over time. Comparatively, other benchmarking approaches have a $O(n)$ complexity in the noise addition scenario. Despite our slightly higher computational complexity, efficiency is achieved by concurrently handling all appliances during noise addition and communicating the result only once. Our approach outperformed other benchmarking methods in utility performance, showcasing superior hit rates on average.

VI. Conclusion

We proposed LDP–$Energy$, a novel LDP method for sharing ED data. LDP–$Energy$ utilizes RR and OUE to discretize and randomize the data through a unique LDP modeling technique for processing ED data. Additionally, LDP–$Energy$ incorporates a sliding window technique to accommodate the streaming nature of the data. In our use-case analysis, LDP–$Energy$ effectively revealed top-k appliance usage patterns and the appliances list from household energy data, maintaining high privacy (e.g., ε = 10) with 15 appliances. The approach demonstrated a hit rate of 8 out of 10 (average 5.7 out of 10), showcasing high accuracy. It ensures privacy and utility for individual users, even with small datasets and strict privacy constraints. This trade-off is acceptable in the real world, as energy service providers with extensive user bases can maintain satisfactory utility despite privacy budget constraints. However, challenges include the need for more users to enhance utility, generalizability across diverse smart home environments, and concerns about the impact of long-term data sharing on utility. Future work aims to address these limitations. Besides, the scalability of our proposed work will be tested further, considering various applications beyond energy disaggregation, such as network usage patterns and environmental sensing data, as future work.

Acknowledgement

The work has been supported by the Cyber Security Research Centre Limited whose activities are partially funded by the Australian Government's Cooperative Research Centres Program.

References

[1] Marks *et al.*, "Differential privacy applied to smart meters: a mapping study," in *Proceedings of the 36th Annual ACM Symposium on Applied Computing*, pp. 761–770, 2021.
[2] Iqbal *et al.*, "A critical review of state-of-the-art non-intrusive load monitoring datasets," *Electric Power Systems Research*, vol. 192, 2021.
[3] J. Kröger, "Unexpected inferences from sensor data: a hidden privacy threat in the internet of things," in *IFIP International Internet of Things Conference*, pp. 147–159, Springer, 2018.
[4] Dwork *et al.*, "Calibrating noise to sensitivity in private data analysis," in *Theory of cryptography conference*, pp. 265–284, Springer, 2006.
[5] Gai *et al.*, "An efficient data aggregation scheme with local differential privacy in smart grid," *Digital Communications and Networks*, 2022.
[6] Liu *et al.*, "Gdp vs. ldp: A survey from the perspective of information-theoretic channel," *Entropy*, vol. 24, no. 3, p. 430, 2022.
[7] Hassan *et al.*, "Differential privacy for renewable energy resources based smart metering," *Journal of Parallel and Distributed Computing*, 2019.
[8] Hassan *et al.*, "Performance evaluation of differential privacy mechanisms in blockchain based smart metering," *arXiv preprint arXiv:2007.09802*, 2020.
[9] Eibl *et al.*, "Differential privacy for real smart metering data," *Computer Science-Research and Development*, vol. 32, no. 1, pp. 173–182, 2017.
[10] Erlingsson *et al.*, "Rappor: Randomized aggregatable privacy-preserving ordinal response," in *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*, pp. 1054–1067, 2014.
[11] Ren *et al.*, "Ldp-ids: Local differential privacy for infinite data streams," in *Proceedings of the 2022 International Conference on Management of Data*, SIGMOD '22, p. 1064–1077, 2022.
[12] Varodayan *et al.*, "Smart meter privacy using a rechargeable battery: Minimizing the rate of information leakage," in *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2011.
[13] Hossain *et al.*, "Cost-friendly differential privacy of smart meters using energy storage and harvesting devices," *IEEE Transactions on Services Computing*, 2021.
[14] Asghar *et al.*, "Smart meter data privacy: A survey," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2820–2835, 2017.
[15] Wang *et al.*, "Locally differentially private protocols for frequency estimation," in *26th USENIX Security Symposium*, 2017.
[16] Wang *et al.*, "Using randomized response for differential privacy preserving data collection.," in *EDBT/ICDT Workshops*, vol. 1558, 2016.
[17] Goddard *et al.*, "Ideal household energy dataset," 2020.
[18] Maharana *et al.*, "A review: Data pre-processing and data augmentation techniques," *Global Transitions Proceedings*, 2022.
[19] Stadler *et al.*, "Synthetic data–anonymisation groundhog day," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022.
[20] McKight *et al.*, "Kruskal-wallis test," *The corsini encyclopedia of psychology*, pp. 1–1, 2010.
[21] Won *et al.*, "A survey on ownership of home appliances and electric energy consumption status according to the number of household member," *Applied Mechanics and Materials*, vol. 672, 2014.
[22] Apple, "Apple differential privacy technical overview." https://www.apple.com/privacy/docs/Differential_Privacy_Overview.pdf, 2023.
[23] Kellaris *et al.*, "Differentially private event sequences over infinite streams," *Proceedings of the VLDB Endowment*, vol. 7, no. 12, 2014.

Authorized licensed use limited to: University of Western Australia. Downloaded on July 11,2026 at 19:08:58 UTC from IEEE Xplore. Restrictions apply.

APPENDIX

A. Randomization Theorm

The length of B is $m = d \times n$, where d represents the number of energy levels and n represents the number of appliances.

Theorem 1. *Let v_1 and v_2 be any bit positions of any two binary vectors b_1 and b_2, respectively, and let B be a d-bit binary string output. When the probability of $B[v_1]$ being 1 given v_1 ($\Pr[B[v_1] = 1|v_1]$) is $\frac{1}{2}$, and the probability of $B[v_2]$ being 0 given v_1 ($\Pr[B[v_2] = 0|v_1]$) is $\frac{e^{\frac{\varepsilon}{2n/2}}}{1+e^{\frac{\varepsilon}{2n/2}}}$], then the randomization model provides ε-LDP.*

Proof.

$$\begin{aligned}
\frac{\Pr[B|v_1]}{\Pr[B|v_2]} &= \frac{\prod_{i\varepsilon[d]} \Pr[B[i]|v_1]}{\prod_{i\varepsilon[d]} \Pr[B[i]|v_2]} \\
&\leq \left[\frac{\Pr[B[v_1] = 1|v_1] \Pr[B[v_2] = 0|v_1]}{\Pr[B[v_1] = 1|v_2] \Pr[B[v_2] = 0|v_2]}\right]^{2n/2} \\
&= \left(\frac{1/2}{\frac{1}{1+e^{\frac{\varepsilon}{2n/2}}}} \cdot \frac{\left(\frac{e^{\frac{\varepsilon}{2n/2}}}{1+e^{\frac{\varepsilon}{2n/2}}}\right)}{1/2}\right)^{2n/2} \\
&= e^{\varepsilon}
\end{aligned}$$

□

B. Adaptive Budget Division Methods

1) Naive methods: Various adaptive budget methods for w-event privacy include *LDP Budget Uniform (LBU)* and *LDP Sampling (LSP)* [11]. LBU uniformly distributes the budget across timestamps but may be less efficient for larger windows. LSP allocates the entire privacy budget ε to a single timestamp in a window of size w, approximating the remaining $(w-1)$ timestamps with perturbed values. However, LSP may not accurately track changing data stream patterns, potentially leading to errors.

2) Local Budget Distribution (LBD): LBD [11] is a method that was initially proposed for global DP in [23]. In contrast to the naive methods, LBD uses statistical dissimilarity metrics to help identify the best approach for either approximating current statistics using past data or adding noise to current statistics when publishing them.

The LBD approach consists of three components: private dissimilarity calculation, private strategy determination, and privacy budget allocation [11]. All calculations occur on the user side and utilize true data to derive intermediate steps. In the context below, ε represents the privacy budget, w denotes the window size, and ε_{rm} is the remaining publication budget at the current timestamp.

In private dissimilarity calculation, the dissimilarity value (*dis*) is determined by comparing the current perturbed value $\bar{c}_t$ (with a fixed dissimilarity budget $\varepsilon_{t,1} = \varepsilon/(2 \times w)$) and the last release using Mean Absolute Error (MAE). For private strategy determination, a budget $\varepsilon_{t,2}$ is allocated for possibility of publishing perturbed data, $\bar{c}t, 2$, with a potential publication budget $\varepsilon_{t,2} = \varepsilon_{rm}/2$). This budget is determined in the privacy budget allocation component, detailed in Algorithm 3, Sub Mechanism $M_{t,2}$. The private strategy determination algorithm uses a decision-making process that compares dissimilarity (*dis*) and potential publication error (*err*) to decide whether to employ the approximation strategy (directly publishing the last released value without using $\varepsilon_{t,2}$) or to publish the current data with perturbation using $\varepsilon_{t,2}$.

In the publication budget allocation component, the LBD approach distributes the entire budget, ε, in a time window evenly between two parts: private dissimilarity estimation and private strategy determination, with dissimilarity and publication budget being $\varepsilon/2$ each ($\sum_{i=t-w+1}^{t} \varepsilon_{i,1} = \sum_{i=t-w+1}^{t} \varepsilon_{i,2} = \varepsilon/2$). Further, the dissimilarity budget is distributed equally among all timestamps within the window, with each timestamp being allocated a budget of $\varepsilon_{i,1} = \varepsilon/(2 \times w)$.

The publication budget $\varepsilon_{i,2}$ is determined using a cost-effective privacy budget allocation strategy, where it is distributed in an exponentially decreasing manner among timestamps. As some publications are skipped via approximation, the remaining privacy budget (ε_{rm}) at each current timestamp is calculated by subtracting the dissimilarity and publication budget spent in a window from the total privacy budget. Next, half of the remaining budget, ε_{rm}, is allocated as the publication budget, $\varepsilon_{i,2}$, and potential publication error (*err*) is calculated. If the current data is selected for publication, the publication budget $\varepsilon_{i,2}$ will be applied in the current timestamp; otherwise, it will be set to 0 for future use.

3) Local Budget Absorption (LBA): LBD and LBA have similar components for calculating private dissimilarity and determining private strategy. However, their methods for dynamically allocating the publication budget of $\varepsilon_{i,2}$ over the stream differ. The LBA approach initially evenly distributes the publication budget across all timestamps. If the current data isn't published, the allocated budget for that timestamp becomes available for future use. However, when data is published at a specific timestamp, it absorbs all available budget accumulated from previous skipped publications. This strategy aims to enhance utility [11], [23]. Absorbing the budget for the current publication nullifies an equivalent amount from succeeding timestamps, preventing the total budget from exceeding the maximum ε. The determination of publication budget $\varepsilon_{i,2}$ is critical, involving decisions on absorption or nullification, which subsequently influences the calculation of potential publication error (*err*). After computing potential publication error (*err*) with $\varepsilon_{i,2}$, the optimal strategy—either publication or approximation—is chosen by comparing it with dissimilarity *dis*, akin to the LBD mechanism. Algorithm 3 outlines the steps of adaptive budget division methods, namely the LBD and LBA algorithms.

C. Utility Analysis

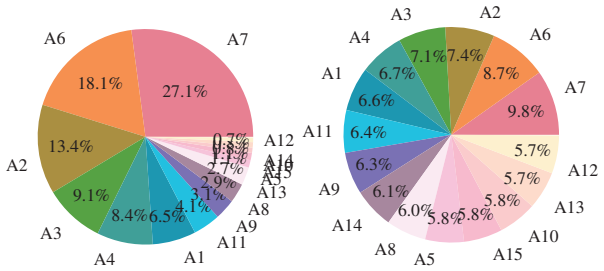
1) More Service Provider-Side Application of Perturbed Data: We examined the impact of individual appliances on overall energy usage during the given time period, as shown in Figures 5a and 5b. Although true and perturbed data percentages differ in both scenarios, we observe a consistent

Authorized licensed use limited to: University of Western Australia. Downloaded on July 11,2026 at 19:08:58 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: The algorithm for adaptive budget division methods: LBD and LBA

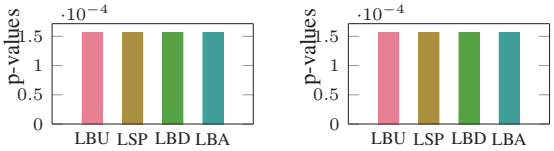
Input: Total privacy budget ε, window size w, current c_t, previous releases $\{r_1, r_2, \ldots, r_l\}$
Output: Released perturbed data

1 *Approach 1: Local budget distribution*
2 Initialize $c_t = <0, \ldots, 0>^d$;
3 **for** *each timestamp* t **do**
4 `// Sub Mechanism` $M_{t,1}$
5 Set $\varepsilon_{t,1} = \varepsilon/(2w)$;
6 Perturb c_t using $\varepsilon_{t,1} \rightarrow \bar{c}_{t,1}$
7 Select lastly published release r_l from $\{r_1, r_2, \ldots, r_l\}$
8 Calculate $dis = \frac{1}{d}\sum_{k=1}^{d}(\bar{c}_{t,1}[k] - r_l[k])^2$
9 `// Sub Mechanism` $M_{t,2}$
10 Calculate $\varepsilon_{rm} = \varepsilon/2$ - $\sum_{i=t-w+1}^{t-1}\varepsilon_{i,2}$;
11 Set $\varepsilon_{t,2} = \varepsilon_{rm}/2$;
12 Perturb c_t using $\varepsilon_{t,2} \rightarrow \bar{c}_{t,2}$
13 Calculate $err = \frac{1}{d}\sum_{k=1}^{d}(\bar{c}_{t,2}[k]] - c_t[k])^2$;
14 `// Sub Mechanism` $M_{t,3}$
15 **if** $dis > err$ **then**
16 `// Publication strategy`
17 **return** $r_t = \bar{c}_{t,2}$ (perturbed using $\varepsilon_{t,2}$)
18 **else**
19 `// Approximation strategy`
20 **return** $r_t = r_l$; set $\varepsilon_{t,2} = 0$
21 **end if**
22 **end for**
23 *Approach 2: Local budget absorption*
24 Initialize $c_t = <0, \ldots, 0>^d$; last pub. timestamp $l = 0$, $\varepsilon_{l,2} = 0$;
25 **for** *each timestamp* t **do**
26 `// Sub Mechanism` $M_{t,1}$
27 Same as lines 5 to 8
28 `// Sub Mechanism` $M_{t,2}$
29 Calculate $t_N = \frac{\varepsilon_{l,2}}{\varepsilon/(2w)} - 1$
30 **if** $t - l \leq t_N$ **then**
31 **return** $r_t = r_l$; set $\varepsilon_{t,2} = 0$
32 **else**
33 Calculate $t_A = t - (l + t_N)$;
34 Set $\varepsilon_{t,2} = \varepsilon/(2w) \times \min(t_A, w)$;
35 Randomize the current data c_t with $\varepsilon_{t,2} \rightarrow \bar{c}_{t,2}$
36 Calculate $err = \frac{1}{d}\sum_{k=1}^{d}(\bar{c}_{t,2}[k]] - c_t[k])^2$
37 `// Sub Mechanism` $M_{t,3}$
38 **if** $dis > err$ **then**
39 `// Publication strategy`
40 **return** $r_t = \bar{c}_{t,2}$ (perturbed using $\varepsilon_{t,2}$); set $l = t$
41 **else**
42 `// Approximation strategy`
43 **return** $r_t = r_l$; set $\varepsilon_{t,2} = 0$
44 **end if**
45 **end if**
46 **end for**



(a) True data (b) Perturbed data

Figure 5: Service provider-side application on IDEAL dataset: Impact of individual appliances on overall energy usage



(a) Uniform distribution (b) Normal distribution

Figure 6: The Kruskal–Wallis similarity test p values for each appliance in uniform and normal distributions

pattern. For instance, in the pie chart, appliances 'A7', 'A6', and 'A2' have a greater impact on total energy usage, while appliances 'A5' and 'A10' have a lesser impact. This pattern remains evident even with perturbed data.

2) More Experimental Results on LDP–Energy Using Synthetic Data: In evaluating *LDP–Energy*'s utility for various distributions (Figure 6), LBU, LSP, LBD, and LBA methods demonstrated similar average performance in uniform and normal distributions.

D. Computational Complexity Analysis

We evaluate the computational complexity of our approach in terms of time requirements using the notation (O) to describe the algorithm's growth rate. At the users' end, smart meters perform real number energy usage mapping, encoding to predefined range levels, and budget calculations involving addition, subtraction, and multiplication in iterative loops for all appliances over time. Part I of the algorithm maps the energy usage of appliances to given range levels. According to Algorithm 1, the overall computational complexity is $O(n^3)$, representing the product of the complexities of the time dimension $O(t)$, the appliance dimension $O(n)$, and the range levels dimension $O(d)$. However, focusing on a single timestamp, the computational complexity for this step is $O(n) \times O(d)$, indicating a quadratic growth rate for a specific timestamp. This complexity is quadratic because it involves iterating over n appliances and performing operations related to d range levels.

Part II of the algorithm involves encoding and creating a flattened vector with a time complexity of $O(n)$. This indicates a linear growth rate, as the time taken for this step increases linearly with the number of appliances (n). Part III of the algorithm involves iterating over timestamps. However, the time taken for each iteration of the loop does not depend on the input size. Therefore, the computational complexity for this part is $O(1)$, indicating constant computational complexity. So, on the client side, the overall complexity is $O(n^2)$.

Service providers handle the aggregation part; the operations involve decoding and addition over multiple users. The computational complexity for this part can be represented as $O(k)$, where k represents the number of users.

Authorized licensed use limited to: University of Western Australia. Downloaded on July 11,2026 at 19:08:58 UTC from IEEE Xplore. Restrictions apply.